

SYSTEM GENERATION PROMPT: PRODUCTION-READY MULTI-PAGE APPAREL E-COMMERCE & DISPATCH SYSTEM

You are a Senior Principal Software Architect, Visual UX/UI Designer, and Lead Full-Stack Engineer. Your objective is to build a highly secure, completely integrated, multi-page Men's Dress Shop E-Commerce and Order Management platform from scratch. The codebase must have zero placeholder logic, zero compilation errors, absolute visual polish, and flawless device-agnostic responsiveness.

1. Technical Stack, Security Policies & Multi-Page Layout Constraints

- **Database:** MySQL 8.x (Write an optimized, self-contained, enterprise-grade `schema.sql` with explicit data types, lengths, indexes, and audit fields).
- **Backend Core:** Spring Boot 3.2.x, Java 17, Maven (Modular structure: Config, Entity, Repository, Service, Controller, DTO, Exception).
- **Token Lifecycle & Storage Policies (Strict XSS Mitigation):**
 - **Access Tokens:** Must be short-lived (exactly 15-minute TTL) and stored only in volatile JavaScript memory (or `sessionStorage` strictly bound by a Content Security Policy). It must **never** be stored in `localStorage`. Access tokens are passed dynamically in the `Authorization: Bearer <token>` HTTP header by the client.
 - **Refresh Tokens:** Must be stored securely in `HttpOnly`, `Secure`, `SameSite=Strict` (or `SameSite=Lax` explicitly mapped on third-party redirection callbacks), TLS-only cookies with token rotation enabled. Every token refresh transaction must invalidate the old token and issue a new, cryptographically rotated pair.
 - **Client Expiry & Silent Refresh Loop:**
 - The client-side JavaScript across all multi-page HTML files must implement a global fetch wrapper.
 - **Silent Refresh Sequence:**

```
Global Client Fetch Wrapper
├─ [1] Intercepts Response
├─ [2] If Status is HTTP 401 (Token Expired)
│   ├── A. Push failed request to dynamic retryQueue array
│   ├── B. If NOT currently refreshing:
│   │   ├── Set refreshing flag = true
│   │   ├── Call refreshing route: POST /api/auth/refresh
│   │   ├── If Refresh successful:
│   │   │   ├── Capture new accessToken from JSON response body
│   │   │   ├── Re-verify active volatile memory variables
│   │   │   └── Iterate retryQueue, resubmitting with new Author
│   │   └── If Refresh fails (HTTP 401 or expired refresh cookie)
│   │       └── Instantly redirect client to auth.html
│   └── C. Wait for ongoing refresh to complete and retry queued re
└─ [3] Return original response if status is successful (HTTP 2xx)
```

- **CORS & CSRF Safeguards:**

Idempotency-Key , X-XSRF-TOKEN) and explicitly validate incoming origins against a production domain whitelist.

- **Double-Submit Cookie CSRF Prevention:** The backend must issue a standard cookie named `XSRF-TOKEN` with `Secure; SameSite=Strict` flags. The client-side JavaScript must parse the token value from `document.cookie` (using a regex extractor) and attach it as the `X-XSRF-TOKEN` HTTP header for all state-changing operations (POST, PUT, DELETE).
- **Frontend Client:** Multi-Page Architecture (MPA) utilizing **Tailwind CSS**, custom native modern CSS transitions, and native Vanilla JS `fetch()` to interact with the Spring Boot REST API.
- **The Required File Directory Matrix:**
 1. `index.html` (Landing page with a filterable clothing catalog)
 2. `auth.html` (Unified forms for Login, Registration, and 6-digit local OTP verification)
 3. `product-details.html` (Rich image layouts, size grids, color swatches, and active customer reviews)
 4. `cart.html` (Persistent cart list with dynamic quantities and instant totals calculation)
 5. `checkout.html` (Profile-autofilled address fields, simulated card/UPI Razorpay portal, or COD choice)
 6. `orders.html` (Customer history panel, 4-step tracking stepper, 3-day cancellation controls, 1-day damaged item return requests)
 7. `admin-dashboard.html` (Admin-only command center displaying gross shop metrics, active shipments, user directories, live dispatcher tracking toggles, and return approval controllers)

2. Technical E-Commerce Flows, Safety, & System Contracts

A. Customer Authentication, Registration, & Secure OTP Flow

- **Registration Inputs:** Username, Email, Password, Phone Number, Mailing Address (Street, City, State, Country), and Date of Birth (DOB).
- **Security Check:** Password must be hashed via BCrypt before database persistence. Account initial verification status is `is_verified = FALSE`.
- **Hashed OTP Generation & Storage:**
 - System generates a 6-digit numeric verification OTP code (C_{otp}) and stores it securely using a cryptographic hash (HMAC-SHA256 with a secure pepper).
 - Establish a strict 5-minute expiration block:

$$t_{expiry} = t_{current} + 5 \text{ minutes}$$

- OTP matching verification must be done using a constant-time comparison algorithm to prevent side-channel timing attacks.
- **Rate Limiting & Attempt Gating:**
 - Enforce a maximum of 3 OTP generation requests per hour per user/IP.
 - Enforce a strict 60-second resend cooldown window.

invalidate the OTP instantly and lock further validation attempts until a new OTP is requested.

- **Production Safety Gating (CI/CD Protection):**

- The backend must **only** return the plaintext verification code inside a `"debug_otp"` field in the JSON response payload if the environment is explicitly configured as a development profile (e.g., `spring.profiles.active=dev` or `app.security.debug-otp-enabled=true`).
- **CI Build-Time Assertions:** Include a unit test/build assertion that actively checks if `app.security.debug-otp-enabled` is false and that the `"debug_otp"` payload generator throws an exception when executing the `prod` build profile. The build must fail if this security check fails.
- **Password Reset Flow:** Uses a cryptographically random, secure UUID token stored in the database as a SHA-256 hash. The reset token must have a strict 1-hour TTL, be restricted to single-use, and be invalidated immediately upon consumption.

B. Apparel Catalog, Pagination, & Live Reviews Flow

- **Dynamic Pagination & Filtering:** Protect database resources by enforcing strict validation on pagination parameters (`page` , `size`) across all public listings and admin feeds.
 - Default page size: 25
 - Maximum permitted page size: 100 (Any request exceeding `size=100` must return `HTTP 400 Bad Request`).
- **Apparel Attributes:** Expose detailed clothing sizing grids (S, M, L, XL, XXL) and color swatch bubbles (Navy Blue, Charcoal Gray, Jet Black, Emerald Green).
- **Apparel Reviews & Stars:** Display chronological user feedback with star elements (1 to 5 stars) alongside user comments.

C. Shopping Cart & "Buy All" Mechanics

- **Cart Interactions:** Cart page reads local storage arrays and displays item breakdowns with `+` and `-` quantity adjustment triggers.
- **Buy All Checkout Trigger:** Consolidates cart arrays, lock values, and carries them to `checkout.html` on click.

D. Concurrency-Safe Checkout & Dynamic Invoicing

- **Dynamic Invoicing & Taxation:** Recalculate total invoice figures dynamically:

$$\text{Total Invoice Amount} = \sum_{i=1}^n (\text{Price}_{\text{item } i} \times \text{Quantity}_i) + \text{Sales Tax} + \text{Shipping Cost}$$

- **Sales Tax:** Explicit 12% sales tax applied to subtotal.
- **Shipping rules:** Apply flat rate of \$15.00, or auto-set to \$0.00 (FREE) if subtotal exceeds \$150.00. Use `DECIMAL(10,2)` for all monetary math.
- **Inventory Concurrency & Thread Safety:**
 - Checkout stock reductions must be fully transactional (`@Transactional(isolation = Isolation.REPEATABLE_READ)`).

product entity. If an `ObjectOptimisticLockingFailureException` occurs, execute an automatic retry loop retrying up to 3 times with exponential backoff delay intervals (100ms, then 200ms, then 400ms) before failing.

- **Fallback Error State:** If retries fail or database stock goes below zero, return `HTTP 409 Conflict` with the system error code `"CONFLICT_STOCK"`.
- **Pessimistic Locking Hotspots:** For ultra-high write hotspots, configure select write endpoints to acquire a pessimistic write lock (`SELECT ... FOR UPDATE` via `@Lock(LockModeType.PESSIMISTIC_WRITE)`) on the catalog inventory record.
- **Transactional Idempotency Key:**
 - Require a unique, client-generated `Idempotency-Key` header for both `/api/orders/checkout` and `/api/payments/create-payment-intent` endpoints.
 - Persist keys in an `idempotent_requests` database table (with columns: `key_hash` `CHAR(64)` `PRIMARY KEY` using SHA-256 hashing, `response_payload` `VARCHAR(10000)` mapping a maximum cached payload of 10KB, and `created_at` `TIMESTAMP`). If a request with an identical key is received within a 24-hour window, return the cached payload immediately without re-processing.

E. Customer Tracking Timeline & Delivery Calibration

- **Delivery Arrival Calculations:** Calculate arrival date dynamically by stamping:

$$D_{delivery} = D_{order} + 4 \text{ days}$$

- **Progress Stepper Tracking:** Display read-only tracking steps: `Order Received` —► `Dispatched` —► `Out for Delivery` —► `Delivered`

F. Post-Purchase 3-Day Cancellation & 1-Day Damaged Return Rules

- **The 3-Day Cancellation Window:**

- Permit cancellations if:

$$t_{current} - t_{order} \leq 3 \text{ days}$$

- On trigger: Set order status to `CANCELLED` within a transaction, and restore catalog inventory stock levels securely using locking mechanics.
- **Idempotent Refund Execution:** If `payment_method` is `RAZORPAY` and status is `PAID`, trigger a simulated gateway refund, store a unique `refund_id` to prevent duplicate processing, and update payment status to `REFUNDED`. Wrap refund calculations + DB updates in a single transaction block. If `COD`, update payment status directly to `CANCELLED`.
- **The 1-Day Damaged Product Return Window:**
 - Allow return requests for damaged products if order status is `DELIVERED` and:

$$t_{current} - t_{actual_delivery} \leq 1 \text{ day}$$

- On trigger: Set order status to `RETURN_REQUESTED`.

RETURNED , restocking items, and processing refunds if paid online.

G. Admin Operations Dashboard Flow

- **Master Dispatch Feed:** Admin views name, phone, address, items, amounts, and tracking state. Contains pagination controls to handle large volumes.
- **Status Change dispatcher:** Admin clicks to advance order tracking states.
- **Delivery Complete Event:** Advancing status to `DELIVERED` stamps the `actual_delivery_date` with the system timestamp. If payment was `COD` , updates `payment_status` automatically to `COD_COLLECTED` .

3. Premium UI/UX, Accessibility, and Responsive Layouts Guidelines

To achieve a clean, high-end, responsive, and compliant design, apply these guidelines across all 7 HTML files:

- **Typography Stack:**
 - Main headers, brand logos, and prices must use `Playfair Display` serif font for a high-end luxury editorial look.
 - Body copy, buttons, forms, and tables must use `Plus Jakarta Sans` sans-serif font for crisp readability.
- **Mobile-First Responsive Framework:**
 - Build layouts defensively utilizing Tailwind's responsive prefixes (`sm: , md: , lg: , xl:`) so that margins, column spans, padding, and text scaling morph perfectly depending on the viewport width.
 - Layout structures must gracefully compress. Multi-column grids must dynamically collapse into single-column layouts on small screens (e.g., `grid-cols-1 md:grid-cols-2 lg:grid-cols-3`).
- **Mobile Touch Target & Accessibility Safety (WCAG 2.1 AA Compliance):**
 - All interactive elements (buttons, size selectors, color swatches, form inputs, links, navigation tabs, and cross-closing buttons) must be designed with ample touching clearances. Target areas must have a minimum dimensions footprint of **44px x 44px** to ensure seamless tapping on mobile screens and to avoid accidental adjacent clicks.
 - Maintain a minimum color contrast ratio of **4.5:1** for all readable text.
 - Modals must implement active keyboard focus traps (preventing users from tabbing outside active boundaries) and programmatically restore focus to the originating trigger element upon close.
 - Toasts must be output using standard non-disruptive announcements: `<div aria-live="polite">` .
- **Responsive Navigation & Fluid UI Elements:**
 - Navigation bars must degrade gracefully into compact mobile nav drawers or toggle sheets.
 - Prevent horizontal viewport page breakage. Set absolute safety margins using structural wraps (`w-full overflow-hidden`).
- **Responsive Admin Dispatch Feed Table:**
 - In `admin-dashboard.html` , wrap data tables in dynamic horizontal scroll views (`overflow-x-auto`) or reflow the layout into stacked interactive orders cards when viewed on displays under 768px wide.

- Standard modals must feature custom transition entry/exit animations (fade-in, slide-up, backdrop-blur transitions).
- Include these custom CSS transitions in a common embedded `<style>` block:

```
@keyframes modalFadeIn {
  from { opacity: 0; transform: scale(0.95) translateY(10px); }
  to { opacity: 1; transform: scale(1) translateY(0); }
}
@keyframes overlayFadeIn {
  from { opacity: 0; backdrop-filter: blur(0px); }
  to { opacity: 1; backdrop-filter: blur(4px); }
}
.animate-modal { animation: modalFadeIn 0.3s cubic-bezier(0.16, 1, 0.3, 1) }
.animate-overlay { animation: overlayFadeIn 0.25s ease-out forwards; }
```

- **Elegant Non-blocking Toast Alerts:** Build a custom screen-overlay alert popup that floats in from the bottom right corner of the active page. It should slide up with a subtle hover drop-shadow and use progress timers.
- **Interactive Step Trackers:** The 4-step progress stepper inside `orders.html` must feature pulse indicator loops (`animate-pulse`) around the active tracking stage, with a glowing fill line showing completed milestones. On small mobile displays, the timeline must transition to a vertical layout to prevent horizontal compression.

4. Database Schema Specifications (`schema.sql`)

Generate a robust, relational, optimized database schema structure:

1. **Database Enums Standardization:** Enforce explicit VARCHAR check constraints to ensure flexible future database migrations without structural table locking.
 - `order_status` Values: ORDER_RECEIVED , DISPATCHED , OUT_FOR_DELIVERY , DELIVERED , CANCELLED , RETURN_REQUESTED , RETURNED
 - `payment_status` Values: PENDING_PAY , PAID , COD_COLLECTED , REFUNDED , FAILED , CANCELLED
 - `payment_method` Values: RAZORPAY , COD
2. **Strict Column Declarations & Canonical Timezones:**
 - Use `DECIMAL(10,2)` for all prices, subtotals, taxes, and totals.
 - Use `INT` or `BIGINT` for quantities.
 - **UTC Timezone Synchronization:** Ensure the MySQL server timezone configuration is explicitly mapped to UTC. All database timestamp columns must store transactions in UTC format.
 - Timestamps returned in API JSON responses must always utilize the ISO-8601 UTC standard formatting: `YYYY-MM-DDTHH:mm:ssZ` (e.g., `2026-05-22T13:49:00Z`).
3. **Optimistic Locking & Audit Columns:**
 - Every primary entity table (specifically `products` and `orders`) must contain a `version` column (defaulting to 0) to support optimistic locking.

updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP) on critical operational tables (users , orders , products).

4. **Optimized Indexes:** Add indexing keys on search-heavy lookup columns:

- Unique indexes on users.email and users.username .
- Index on orders.user_id and orders.status .
- Index on products.category_id .
- Index on order_items.order_id .

Ensure fully functional, formatted database seed queries introducing your categories, items, and initial accounts.

5. Backend Integration, Exception Handling, & API Rules

- **Prevent JSON Infinite Recursion Loops:** Implement @JsonIgnore or use custom DTO structures in the entities to block infinite nested loops during REST responses (specifically within bidirectional mappings between entities, e.g., Order and OrderItem).
- **CORS Credentials & Headers Security:** In the Spring Boot config layer, ensure that the CORS configuration permits headers like Authorization and Content-Type for cross-origin requests, aligned securely with strict host whitelist validation.
- **Central Exception Handling & Response Components:**
 - Implement a @ControllerAdvice global handler mapping business exceptions to structured JSON API error payloads. Every exception response returned by the controllers must conform to this schema:

```
{
  "timestamp": "2026-05-22T13:49:00Z",
  "code": "INSUFFICIENT_STOCK",
  "message": "Required quantities not available in catalog inventory.",
  "correlationId": "err_1a2b3c4d5e",
  "details": ["Requested item ID 5 stock level is 3, requested 5."]
}
```

- **OpenAPI 3.0 Documentation:** Include Swagger API documentation or provide an inline, fully detailed openapi.yaml mapping out endpoints, input fields, request bodies, response models, status codes, and security requirements. Ensure the standard error schema above is defined in components and referenced properly across all controllers.
- **Code Completeness:** No mock configurations, shortcuts, or commented placeholders.

6. Observability, Deployment, & Operations

- **Database Migrations:** Include configurations to support automated DB migrations using Flyway or Liquibase.
- **Observability & Logging:** Implement structured JSON logging on the backend. Every API request must generate or pass along a correlation ID (X-Correlation-ID header) to trace logs from the frontend down to the database transactions.
- **Alert thresholds & Retention:**
 - Configure Prometheus metrics and alert manager rules for:

- Repeated negative stock check attempts (exceeding 10 checks per minute).
- OTP verification abuse indicators (exceeding 10 failed attempts from a single IP/hour).
- System logging profiles must maintain a 30-day active retention policy with logs formatted in structured, queryable JSON fields.
- **Deployment Checklist:**
 - Ensure all production secrets (database passwords, JWT signing keys, Razorpay credential keys) are referenced through environment variables (ENV) and never committed as